

**UNIVERSIDAD POLITÉCNICA DE
AGUASCALIENTES**
INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E
INNOVACIÓN DIGITAL



Tarea 6: Automatización de pruebas y análisis de calidad del proyecto integrador

ASIGNATURA

Estándares y Métricas para el Desarrollo de Software

DOCENTE

Dr(c). Raul Alejandro Velasquez Ortiz

PROYECTO INTEGRADOR

Outline - Wiki interna estilo Notion (Open Source maduro)

EQUIPO No. 4

CHAIREZ ALBA OLIVIA LIZBETH - UP240594

ESTRADA RAMÍREZ NOÉ EZEQUIEL - UP240770

FACIO PALOS OMAR - UP240470

JAUREGUI DE LA RIVA ALAN RICARDO - UP240960

RUBIO VIRAMONTES BRIAN ODIN - UP250004

GRUPO

TIID05B

Repositorio Git: <https://github.com/UP240594/outline-sqa-docs>

Aguascalientes, México 21 de junio de 2026

Índice

1. Introducción	2
2. Desarrollo	3
2.1. Investigación: frameworks de automatización E2E	3
2.2. Investigación: análisis estático de código	5
2.3. Implementación de pruebas E2E con Selenium	7
2.3.1. Estructura del proyecto de pruebas	7
2.3.2. Page Objects: localizadores y métodos de negocio	9
2.3.3. Casos de prueba: válidos, de frontera y de error	12
2.3.4. Esperas explícitas (prohibido <code>time.sleep</code>)	14
2.3.5. Evidencias de ejecución	14
2.4. Pruebas de Componentes y Reporte de Pruebas	14
2.4.1. Implementación de Pruebas Data-Driven con Pytest	14
2.4.2. Evidencia y Reporte Dinámico de Pruebas	16
2.4.3. Nota Técnica sobre el Entorno de Pruebas y Mitigación Operacional	16
2.5. Integración y CI (Continuous Integration)	18
2.5.1. Análisis de los Pasos del Pipeline	19
2.5.2. Condiciones del Quality Gate y Escenarios de Bloqueo	19
3. Evidencias del Proceso de Configuración y Ejecución	21
4. Resultados del Análisis de Calidad y Conclusiones	22
4.1. Interpretación SQA y Recomendaciones	22
4.2. Plan de remediación de deuda técnica	23
5. Conclusión integral	24
6. Aportes individuales	25

Introducción

El presente documento detalla la investigación e implementación de estrategias de aseguramiento de calidad aplicadas a nuestro proyecto integrador, el cual consiste en el desarrollo de una plataforma tipo Wiki. En el ciclo de vida del desarrollo de software, garantizar la fiabilidad, seguridad y mantenibilidad del código es fundamental, por lo que la adopción de prácticas de evaluación continua, como las pruebas automatizadas y el análisis estático, resultan indispensables para el éxito del producto.

En la primera parte de este trabajo, se presenta una investigación comparativa sobre herramientas modernas para la automatización de pruebas de extremo a extremo (End-to-End o E2E) y plataformas de análisis estático de código. El objetivo de esta fase teórica es evaluar distintas alternativas actuales, contrastando sus características, ventajas, curvas de aprendizaje y compatibilidad, para justificar las opciones más adecuadas según la arquitectura y el *stack* tecnológico de nuestra Wiki.

Posteriormente, se documenta la aplicación práctica de estas herramientas sobre el entorno real de nuestro proyecto. Por un lado, se implementa una suite de pruebas automatizadas E2E utilizando Selenium y `pytest`, estructurada bajo el patrón de diseño *Page Object Model* (POM) e incorporando pruebas basadas en datos (*data-driven*). Por otro lado, se integra SonarQube para ejecutar un escaneo profundo del código fuente de la plataforma, evaluando las dimensiones clave de calidad (fiabilidad, seguridad, mantenibilidad, cobertura y duplicaciones). A partir de los hallazgos técnicos detectados, se propone un plan de remediación priorizado por severidad bajo la filosofía *Clean as You Code*, estableciendo las bases para una mejora continua apoyada en un *pipeline* de Integración Continua (CI).

Desarrollo

Investigación: frameworks de automatización E2E

La automatización de pruebas de extremo a extremo (*end-to-end*, E2E) valida el comportamiento del sistema desde la perspectiva del usuario final, ejercitando la interfaz real contra una aplicación en ejecución (jorgensen2022). Para nuestro proyecto integrador —un wiki colaborativo construido con un frontend en React/TypeScript y un backend en Node.js— la elección del framework determina la velocidad de la retroalimentación, la estabilidad de la *suite* y el costo de mantenimiento a largo plazo. A continuación comparamos las tres herramientas de referencia del ecosistema E2E en 2026: **Selenium WebDriver**, **Cypress** y **Playwright**.

Selenium WebDriver. Es la herramienta más madura del ecosistema y opera sobre el estándar *W3C WebDriver*, comunicándose con cada navegador a través de su *driver* correspondiente (seleniumdocs). Su mayor fortaleza es la amplitud: soporta múltiples lenguajes (Java, Python, C#, JavaScript, Ruby, Kotlin) y la mayor variedad de navegadores, incluyendo motores heredados. La gestión de esperas se realiza mediante esperas explícitas con `WebDriverWait` y condiciones esperadas, lo que evita la fragilidad de las esperas fijas (garcia2022). Como contrapartida, exige más código de andamiaje (*boilerplate*), su curva de aprendizaje es más pronunciada y la ejecución en paralelo depende de infraestructura adicional (Selenium Grid).

Cypress. Adopta una arquitectura distinta: ejecuta el código de prueba *dentro* del propio navegador, en el mismo bucle de eventos de JavaScript que la aplicación bajo prueba (cypressdocs). Esto le otorga latencia de red nula, espera automática (*auto-waiting*) y un depurador con “viaje en el tiempo” que captura el estado del DOM en cada paso, lo que se traduce en una experiencia de desarrollo (DX) superior. Sus limitaciones son consecuencia de esa misma arquitectura: solo admite JavaScript/TypeScript, su soporte de WebKit/Safari es experimental e incompleto, y la ejecución paralela robusta requiere el servicio de pago Cypress Cloud.

Playwright. Desarrollado por Microsoft, automatiza de forma nativa los tres motores principales —Chromium, Firefox y WebKit— y ofrece enlaces oficiales para JavaScript/TypeScript, Python, Java y C# (playwrightdocs). Incorpora espera automática basada en comprobaciones de *actionability* (el elemento debe ser visible, estable, habilitado y no estar obstruido antes de interactuar), contextos de navegador aislados, un *Trace Viewer* para el análisis post-mortem y, de forma destacada, ejecución paralela y *sharding* nativos

sin necesidad de un *grid*. Estas características reducen la cantidad de pruebas inestables (*flaky*) y disminuyen el costo total de propiedad al escalar la *suite*.

Tabla comparativa. La Tabla 1 sintetiza la comparación según diez criterios técnicos relevantes para nuestro contexto.

Cuadro 1: Comparación de frameworks de automatización E2E (2026).

Criterio	Selenium	Cypress	Playwright
Lenguajes soportados	Java, Python, C#, JavaScript, Ruby, Kotlin	Solo JavaScript / TypeScript	JavaScript/TypeScript, Python, Java, C#
Arquitectura	Protocolo W3C WebDriver vía <i>drivers</i> externos	Ejecuta dentro del navegador (mismo bucle de JS)	Comunicación directa con el navegador (WebSocket/CDP)
Curva de aprendizaje	Pronunciada; requiere más <i>boilerplate</i>	Baja para equipos de JS; DX muy amigable	Media; API moderna y <i>codegen</i>
Velocidad / rendimiento	Moderada; sobrecarga de WebDriver	Buena en local; arranque más lento	Alta; la más rápida en ejecución paralela
Manejo de esperas	Explícitas (<code>WebDriverWait</code>); evita esperas fijas	Espera automática integrada	Espera automática por <i>actionability</i>
Soporte de navegadores	El más amplio (incluye motores heredados)	Chromium y Firefox nativos; WebKit experimental	Chromium, Firefox y WebKit nativos
Ejecución en paralelo	Mediante Selenium Grid (infraestructura adicional)	Limitada; paralelismo robusto requiere Cypress Cloud	Nativa, sin <i>grid</i> (<i>workers</i> y <i>sharding</i>)
Pruebas móviles	Nativas vía Appium	No (solo emulación de <i>viewport</i>)	No (solo emulación de dispositivos)
Comunidad / madurez	La más madura; estándar de facto empresarial	Fuerte en <i>frontend</i> JS; comunidad consolidada	Crecimiento más rápido; comunidad en expansión
Licencia	Apache 2.0 (código abierto)	MIT (código abierto; <i>Cloud</i> de pago)	Apache 2.0 (código abierto)

Fuente: elaboración propia con base en la documentación oficial de Selenium, Cypress y Playwright (seleniumdocscypressdocs; playwrightdocs;) y en garcia2022.

Recomendación justificada. Para el contexto de nuestro proyecto —una aplicación web de página única (SPA) con uso intensivo de JavaScript y un *stack* de TypeScript/React/Node.js— la recomendación técnica es **Playwright**. Tres razones lo respaldan: (1) sus enlaces de primera clase para TypeScript se alinean directamente con el lenguaje del proyecto, reduciendo la fricción de adopción; (2) su soporte nativo de WebKit permite cubrir Safari, navegador relevante para un wiki de acceso público que Cypress no cubre de forma fiable; y (3) su ejecución paralela nativa, junto con la espera por *actionability*, ataca directamente el principal problema de las *suites* E2E —la inestabilidad— sin costos de infraestructura adicionales, en línea con el principio de pruebas rápidas y confiables que recomienda aniche2022.

No obstante, conforme a lo trabajado en la Semana 7, la *implementación* práctica (sección 4.3 y siguientes) se realiza con **Selenium** + **pytest** aplicando el patrón *Page Object Model*. Esta diferencia entre la herramienta recomendada y la implementada es deliberada y está prevista por la asignatura: Selenium ofrece el soporte más amplio de lenguajes y navegadores, una base conceptual sólida sobre el estándar WebDriver y un ecosistema empresarial maduro, lo que lo hace idóneo como herramienta de aprendizaje (garcia2022). Las técnicas de diseño de casos (válidos, de frontera y de error) que aplicamos son independientes del framework (jorgensen2022), por lo que la suite construida con Selenium puede migrarse a Playwright en el futuro conservando su valor.

Investigación: análisis estático de código

El análisis estático examina el código fuente *sin ejecutarlo*, identificando defectos a partir de su estructura sintáctica y semántica. Es una práctica de “desplazamiento a la izquierda” (*shift-left*) que detecta problemas en las etapas tempranas del ciclo de vida, cuando corregirlos es más barato. En esta sección comparamos tres herramientas representativas de categorías distintas —**SonarQube**, **Semgrep** y los *linters* **ESLint/PyLint**— y diferenciamos el análisis estático del dinámico.

SonarQube. Es una plataforma de calidad y seguridad de código que cubre más de 30 lenguajes mediante miles de reglas predefinidas (sonarqubedocs). Detecta *bugs* (confiabilidad), vulnerabilidades y *security hotspots* (seguridad) y *code smells* (mantenibilidad), y además mide cobertura, duplicación y complejidad ciclomática y cognitiva. Su característica distintiva son las *Quality Gates*: criterios automáticos de aprobación o rechazo que pueden bloquear la fusión (*merge*) de una rama cuando el código nuevo no cumple los umbrales definidos. Es importante una precisión para la sección 4.4: bajo el modelo *Clean Code* actual, SonarQube define *tres* cualidades de software —Seguridad, Confiabilidad y Mantenibilidad— que, sumadas a las medidas de *Cobertura* y *Duplicación* del tablero, conforman las cinco dimensiones que reporta la herramienta (sonarqubedocs).

Semgrep. Es un motor de análisis estático orientado a la seguridad (SAST) que opera mediante coincidencia de patrones sobre el árbol de sintaxis abstracta, no sobre texto plano. Su fortaleza es la rapidez (escaneos del orden de segundos) y la facilidad para escribir reglas personalizadas en YAML, lo que lo hace popular para hacer cumplir políticas de seguridad específicas. Detecta patrones explotables —inyección SQL, *cross-site scripting* (XSS), credenciales codificadas, deserialización insegura— mediante análisis de contaminación (*taint analysis*). Su limitación es que está enfocado en seguridad: no calcula métricas de mantenibilidad, complejidad, duplicación ni cobertura.

ESLint / Pylint (linters). Representan la capa más rápida y cercana al desarrollador. ESLint analiza JavaScript y TypeScript mediante un sistema de reglas configurable basado en el AST, mientras que Pylint hace lo equivalente para Python. Aplican estilo y convenciones, detectan errores comunes y se ejecutan en tiempo real dentro del IDE. Su limitación es la profundidad: no realizan análisis de seguridad entre archivos ni detectan errores lógicos o de tiempo de ejecución; por ello suelen complementarse con una herramienta SAST o una plataforma de calidad.

Tabla comparativa. La Tabla 2 resume las diferencias. Se incluye CodeClimate (hoy en transición a “qlty”) como referencia adicional de plataforma orientada a la mantenibilidad.

Cuadro 2: Comparación de herramientas de análisis estático (2026).

Criterio	SonarQube	Semgrep	ESLint / Pylint
Categoría	Plataforma de calidad + seguridad	Motor SAST (seguridad)	<i>Linter</i> (estilo + errores básicos)
Defectos que detecta	<i>Bugs</i> , vulnerabilidades, <i>code smells</i> , duplicación	Vulnerabilidades de seguridad y <i>anti-patrones</i>	Estilo, convenciones y errores comunes
Enfoque de detección	Miles de reglas predefinidas por lenguaje	Coincidencia de patrones sobre el AST (reglas en YAML)	Reglas configurables sobre el AST
Métricas de calidad	Sí: complejidad, duplicación, cobertura, deuda técnica	No (solo seguridad)	No (solo reglas de estilo/sintaxis)
Integración CI/CD	<i>Quality Gates</i> que bloquean <i>merges</i>	Rápida; ideal para escaneo en cada <i>pull request</i>	En cada <i>commit</i> ; muy rápida
Lenguajes	30+ lenguajes	35+ lenguajes	ESLint: JS/TS; Pylint: Python
Licencia / costo	<i>Community Edition</i> gratuita; ediciones de pago	Núcleo de código abierto (LGPL); gratis hasta cierto número de contribuyentes	Código abierto (gratuito)

Fuente: elaboración propia con base en la documentación oficial de las herramientas (sonarqubedocs).

Análisis estático frente a análisis dinámico. La diferencia fundamental es la *ejecución*. El análisis estático inspecciona el código sin correrlo y, por ello, puede razonar sobre *todas* las rutas del programa, detectando *code smells*, vulnerabilidades por patrón, complejidad excesiva y duplicación antes de que el software se ejecute siquiera. Su contrapartida son los falsos positivos y la imposibilidad de observar comportamientos que solo emergen en tiempo de ejecución. El análisis dinámico, en cambio, examina el programa *mientras se ejecuta* con entradas concretas: detecta fallos reales, fugas de memoria, cuellos de botella de rendimiento y condiciones de carrera, pero solo sobre las rutas efectivamente ejercitadas (de ahí su riesgo de falsos negativos) y requiere un entorno de ejecución y datos de prueba. De hecho, la propia *suite* E2E con Selenium de la sección 4.3 constituye una

forma de análisis dinámico, pues valida la aplicación en funcionamiento. Ambos enfoques son complementarios y se integran en el Plan de Pruebas de la Unidad II: SonarQube (estático) y la suite de pruebas con pytest/Selenium (dinámico) cubren clases de defectos distintas (aniche2022khorikov2020;).

Justificación para nuestro *stack*. Recomendamos **SonarQube** como herramienta principal de análisis estático para el proyecto por cuatro motivos: (1) analiza de forma nativa JavaScript y TypeScript, los lenguajes de nuestro frontend y backend; (2) unifica en un solo tablero las cinco dimensiones que pide la evaluación (Confiabilidad, Seguridad, Mantenibilidad, Cobertura y Duplicación), facilitando la sección 4.4; (3) sus *Quality Gates* se integran con el pipeline de CI de la sección 4.5 para bloquear fusiones que no cumplan los umbrales; y (4) su *Community Edition* es gratuita y, mediante SonarCloud, el análisis es gratuito para repositorios públicos. Como complemento, conservamos **ESLint** —ya estándar en proyectos de React/Node.js— como capa rápida de *linting* en cada *commit*, y dejamos abierta la incorporación de **Semgrep** si el equipo requiere profundizar en seguridad. Esta estratificación (*linter* en cada *commit*, plataforma de calidad en cada PR) es la práctica recomendada para mantener la salud del código sin sacrificar velocidad (aniche2022).

Implementación de pruebas E2E con Selenium

Sobre un módulo real del proyecto integrador (el wiki Outline) implementamos una *suite* de pruebas de extremo a extremo con **Selenium WebDriver** y **pytest**, aplicando el patrón *Page Object Model* (POM). El POM encapsula, en clases independientes, los localizadores y las interacciones con cada pantalla, de modo que las pruebas expresen intención de negocio y no detalles del DOM; así, un cambio en la interfaz se corrige en un solo lugar y la *suite* resulta más mantenible (garcia2022khorikov2020;). Esta primera parte cubre la arquitectura del proyecto, los Page Objects y el conjunto de pruebas con sus tres tipos de caso, empleando exclusivamente esperas explícitas.

Estructura del proyecto de pruebas

El proyecto separa los Page Objects (`pages/`) de las pruebas (`tests/`) y centraliza la configuración en `conftest.py`, conforme a la organización recomendada para suites con pytest (pytestdocs).

```
e2e_outline/
|-- pages/
|   |-- base_page.py      # Esperas explícitas reutilizables (WebDriverWait)
|   |-- login_page.py     # Page Object 1: autenticacion
|   '-- search_page.py    # Page Object 2: busqueda
```



```

|-- tests/
|   |-- test_login.py      # 5 pruebas (incluye 1 data-driven parametrizada)
|   '-- test_search.py    # 3 pruebas (requieren sesion)
|-- conftest.py           # Fixtures: driver, base_url, authenticated_driver
|-- pytest.ini
'-- requirements.txt

```

Listing 1: Estructura de directorios de la suite E2E.

La clase `BasePage` concentra la lógica de sincronización: todos los Page Objects heredan de ella sus métodos de espera, lo que garantiza que ningún punto de la suite recurra a esperas fijas.

```

1 from selenium.webdriver.common.by import By
2 from selenium.webdriver.support.ui import WebDriverWait
3 from selenium.webdriver.support import expected_conditions as EC
4 from selenium.common.exceptions import TimeoutException
5
6
7 class BasePage:
8     DEFAULT_TIMEOUT = 15 # segundos
9
10    def __init__(self, driver, base_url, timeout=DEFAULT_TIMEOUT):
11        self.driver = driver
12        self.base_url = base_url.rstrip("/")
13        self.wait = WebDriverWait(driver, timeout)
14
15    def open(self, path=""):
16        self.driver.get(f"{self.base_url}/{path.lstrip('/')}")
17        return self
18
19    def visible(self, locator):
20        return self.wait.until(EC.visibility_of_element_located(locator))
21
22    def clickable(self, locator):
23        return self.wait.until(EC.element_to_be_clickable(locator))
24
25    def present(self, locator):
26        return self.wait.until(EC.presence_of_element_located(locator))
27
28    def is_visible(self, locator, timeout=None):
29        w = self.wait if timeout is None else WebDriverWait(self.driver, timeout)
30        try:
31            w.until(EC.visibility_of_element_located(locator))
32            return True
33        except TimeoutException:
34            return False
35

```

```

36     def count(self, locator):
37         return len(self.driver.find_elements(*locator))

```

Listing 2: pages/base_page.py — utilidades de espera explícita.

Page Objects: localizadores y métodos de negocio

Outline está construido con React y *styled-components*, por lo que sus clases CSS son hasheadas e inestables. En consecuencia, los localizadores se definen por *atributo* (*type*, *name*, *placeholder*), por *rol* y por *texto*, nunca por clase (*seleniumdocs*). El primer Page Object modela la pantalla de autenticación.

```

1  from selenium.webdriver.common.by import By
2  from .base_page import BasePage
3
4
5  class LoginPage(BasePage):
6      PATH = "/" # la raíz redirige al login si no hay sesión
7
8      EMAIL_TOGGLE = (By.XPATH,
9                      '//button[contains(., "Email") or contains(., "correo")]')
10     EMAIL_INPUT = (By.CSS_SELECTOR,
11                   'input[type="email"], input[name="email"]')
12     SUBMIT_BUTTON = (By.XPATH,
13                     '//button[@type="submit" or contains(., "Continue") '
14                     'or contains(., "Continuar")]')
15     CONFIRMATION = (By.XPATH,
16                    '//*[contains(., "Check your email") or contains(., "revisa tu correo")]')
17
18     def load(self):
19         return self.open(self.PATH)
20
21     def _reveal_email_field(self):
22         if not self.is_visible(self.EMAIL_INPUT, timeout=3):
23             try:
24                 self.clickable(self.EMAIL_TOGGLE).click()
25             except Exception:
26                 pass
27         return self
28
29     def login_with_email(self, email):
30         self._reveal_email_field()
31         field = self.visible(self.EMAIL_INPUT)
32         field.clear()
33         if email:
34             field.send_keys(email)

```

```

35     self.clickable(self.SUBMIT_BUTTON).click()
36     return self
37
38     def email_field_displayed(self):
39         self._reveal_email_field()
40         return self.is_visible(self.EMAIL_INPUT)
41
42     def confirmation_displayed(self):
43         return self.is_visible(self.CONFIRMATION, timeout=8)
44
45     def still_on_login(self):
46         return self.is_visible(self.EMAIL_INPUT, timeout=3) \
47             or "/login" in self.driver.current_url

```

Listing 3: pages/login_page.py — Page Object de autenticación.

El segundo Page Object modela el módulo de búsqueda. Localiza el campo de búsqueda por atributo, los resultados por su enlace a documentos (/doc/...) y el estado vacío por texto.

```

1  from selenium.webdriver.common.by import By
2  from selenium.webdriver.common.keys import Keys
3  from .base_page import BasePage
4
5
6  class SearchPage(BasePage):
7      PATH = "/search"
8
9      SEARCH_INPUT = (By.CSS_SELECTOR,
10         'input[type="search"], input[placeholder*="Search"]')
11      RESULT_ITEMS = (By.CSS_SELECTOR,
12         'a[href*="/doc/"], [data-testid="search-result"]')
13      EMPTY_STATE = (By.XPATH,
14         '//*[contains(., "No results") or contains(., "Sin resultados")]')
15
16     def load(self):
17         return self.open(self.PATH)
18
19     def search(self, query):
20         field = self.visible(self.SEARCH_INPUT)
21         field.clear()
22         field.send_keys(query)
23         field.send_keys(Keys.ENTER)
24         return self
25
26     def has_results(self):
27         return self.is_visible(self.RESULT_ITEMS, timeout=10)
28

```

```

29     def empty_state_displayed(self):
30         return self.is_visible(self.EMPTY_STATE, timeout=10)

```

Listing 4: pages/search_page.py — Page Object de búsqueda.

La configuración compartida vive en `conftest.py`. La *fixture driver* usa *Selenium Manager* (Selenium 4.6+) para resolver el *driver* automáticamente y evitar choques de versión entre Chrome y `chromedriver`. La *fixture authenticated_driver* inyecta una cookie de sesión para las pruebas que requieren login y se *salta* limpiamente si no se proporciona.

```

1  import os
2  import pytest
3  from selenium import webdriver
4  from selenium.webdriver.chrome.options import Options
5
6
7  @pytest.fixture(scope="session")
8  def base_url():
9      return os.getenv("OUTLINE_BASE_URL", "http://localhost:3000")
10
11
12  @pytest.fixture
13  def driver():
14      options = Options()
15      if os.getenv("HEADLESS", "true").lower() == "true":
16          options.add_argument("--headless=new")
17      options.add_argument("--no-sandbox")
18      options.add_argument("--disable-dev-shm-usage")
19      options.add_argument("--window-size=1280,900")
20      drv = webdriver.Chrome(options=options)
21      drv.set_page_load_timeout(30)
22      yield drv
23      drv.quit()
24
25
26  @pytest.fixture
27  def authenticated_driver(driver, base_url):
28      name = os.getenv("OUTLINE_COOKIE_NAME")
29      value = os.getenv("OUTLINE_COOKIE_VALUE")
30      if not name or not value:
31          pytest.skip("Falta cookie de sesión.")
32      driver.get(base_url) # estar en el dominio antes de add_cookie
33      driver.add_cookie({"name": name, "value": value, "path": "/"})
34      driver.get(base_url)
35      return driver

```

Listing 5: conftest.py — fixtures de pytest.

Casos de prueba: válidos, de frontera y de error

La *suite* contiene ocho pruebas que cubren los tres tipos de caso exigidos. La selección de casos sigue las técnicas de partición de equivalencias y valores de frontera, independientes del framework empleado (aniche2022). La Tabla 3 resume la matriz de pruebas.

Cuadro 3: Matriz de casos de prueba de la suite E2E.

Prueba	Módulo	Tipo de caso
test_login_page_shows_email_field	Login	Válido
test_valid_email_shows_confirmation	Login	Válido
test_invalid_email_is_rejected	Login	Error
test_empty_email_cannot_submit	Login	Frontera / Error
test_login_email_validation_parametrized	Login	Data-driven (válido, frontera y error)
test_search_existing_term_returns_results	Búsqueda	Válido
test_search_nonexistent_term_shows_empty_state	Búsqueda	Error
test_search_single_character_boundary	Búsqueda	Frontera

Las pruebas del módulo de autenticación incluyen una prueba *data-driven* con `@pytest.mark.parametrize` que valida cinco formatos de correo en una sola función.

```
1 import pytest
2 from pages.login_page import LoginPage
3
4 VALID_EMAIL = "qa.test@example.com"
5
6
7 def test_login_page_shows_email_field(driver, base_url):
8     """VÁLIDO: la pantalla de login muestra el campo de correo."""
9     page = LoginPage(driver, base_url).load()
10    assert page.email_field_displayed()
11
12
13 def test_valid_email_shows_confirmation(driver, base_url):
14     """VÁLIDO: un correo bien formado avanza a confirmación."""
15     page = LoginPage(driver, base_url).load()
16     page.login_with_email(VALID_EMAIL)
17     assert page.confirmation_displayed()
18
19
20 def test_invalid_email_is_rejected(driver, base_url):
21     """ERROR: un correo mal formado no debe avanzar."""
22     page = LoginPage(driver, base_url).load()
23     page.login_with_email("esto-no-es-un-correo")
24     assert not page.confirmation_displayed()
25     assert page.still_on_login()
26
```

```

27
28 def test_empty_email_cannot_submit(driver, base_url):
29     """FRONTERA/ERROR: el formulario vacío no debe avanzar."""
30     page = LoginPage(driver, base_url).load()
31     page.login_with_email("")
32     assert not page.confirmation_displayed()
33     assert page.still_on_login()
34
35
36 @pytest.mark.parametrize("email, should_pass", [
37     ("a@b.co", True), # FRONTERA: mínimo válido
38     ("nombre.apellido@upa.edu.mx", True), # VÁLIDO: típico
39     ("sin-arroba.com", False), # ERROR: falta '@'
40     ("@sin-local", False), # ERROR: sin parte local
41     ("", False), # FRONTERA: vacío
42 ])
43 def test_login_email_validation_parametrized(driver, base_url, email, should_pass):
44     """DATA-DRIVEN: valida varios formatos de correo."""
45     page = LoginPage(driver, base_url).load()
46     page.login_with_email(email)
47     if should_pass:
48         assert page.confirmation_displayed()
49     else:
50         assert not page.confirmation_displayed()

```

Listing 6: tests/test_login.py — pruebas de autenticación.

Las pruebas del módulo de búsqueda usan la *fixture* `authenticated_driver` y cubren un caso válido (término existente), uno de error (término inexistente) y uno de frontera (un solo carácter).

```

1 from pages.search_page import SearchPage
2
3 EXISTING_TERM = "welcome" # ajustar a un documento existente
4
5
6 def test_search_existing_term_returns_results(authenticated_driver, base_url):
7     """VÁLIDO: un término existente devuelve resultados."""
8     page = SearchPage(authenticated_driver, base_url).load()
9     page.search(EXISTING_TERM)
10    assert page.has_results()
11
12
13 def test_search_nonexistent_term_shows_empty_state(authenticated_driver, base_url):
14     """ERROR: un término inexistente muestra el estado vacío."""
15     page = SearchPage(authenticated_driver, base_url).load()
16     page.search("zxqwk-termino-inexistente-987654")
17    assert page.empty_state_displayed()

```

```

18
19
20 def test_search_single_character_boundary(authenticated_driver, base_url):
21     """FRONTERA: una búsqueda de un carácter no debe romper la UI."""
22     page = SearchPage(authenticated_driver, base_url).load()
23     page.search("a")
24     assert page.has_results() or page.empty_state_displayed()

```

Listing 7: tests/test_search.py — pruebas de búsqueda.

Esperas explícitas (prohibido `time.sleep`)

Toda la sincronización se resuelve con `WebDriverWait` y `expected_conditions`, centralizados en `BasePage`. Una espera explícita suspende la ejecución hasta que se cumple una condición concreta —por ejemplo, que un elemento sea visible o clickeable— con un tiempo máximo, y continúa en cuanto la condición se satisface. Esto elimina las pausas fijas (`time.sleep`), que o bien ralentizan la suite innecesariamente o bien la vuelven inestable (*flaky*) cuando la aplicación responde más lento de lo previsto (garcia2022seleniumdocs;). En consecuencia, ningún Page Object ni prueba de esta suite utiliza `time.sleep`: las condiciones esperadas (`visibility_of_element_located`, `element_to_be_clickable`, `presence_of_element_located`) cubren todos los puntos de sincronización.

Evidencias de ejecución

La suite se ejecuta y documenta con:

```
pytest --cov=pages --cov-report=xml --html=reporte.html --self-contained-html -v
```

Las evidencias correspondientes —la salida de consola de `pytest -v` y el reporte HTML generado— se incluyen como figuras en la Parte 2 de esta misma sección.

Pruebas de Componentes y Reporte de Pruebas

Implementación de Pruebas Data-Driven con Pytest

Para maximizar la eficiencia y exhaustividad de la suite de pruebas funcionales de la Wiki sin incurrir en duplicación de código, se diseñó e implementó una prueba basada en datos (*Data-Driven Testing*) utilizando el decorador nativo `@pytest.mark.parametrize` de Pytest. Esta técnica metodológica permite desacoplar la lógica de ejecución del flujo de prueba de los datos de entrada, evaluando múltiples escenarios en un único bloque de código ejecutable.

El objetivo principal de esta prueba parametrizada fue validar el comportamiento robusto del motor de búsqueda de la Wiki bajo condiciones variables. Los casos de prueba estructurados abarcan escenarios válidos, de frontera y de manejo de errores, definidos de la siguiente manera:

- **Caso Válido (Artículo Existente):** Verifica que al ingresar un término válido (e.g., “Software Architecture”), el sistema devuelva los resultados correspondientes correctamente.
- **Caso de Frontera (Artículo Inexistente):** Valida la respuesta del sistema ante una cadena alfanumérica sin coincidencias en la base de datos (e.g., “NonExistentArticle123”).
- **Caso de Error (Caracteres Especiales):** Evalúa la tolerancia de la aplicación a fallos e inyecciones básicas mediante caracteres especiales (e.g., “!@#%\$”).
- **Caso de Error (Búsqueda Vacía):** Comprueba que el sistema reaccione de manera controlada ante el envío de una cadena vacía ().

En el Código 8 se expone la implementación formal de la prueba parametrizada conectada con el Page Object Model de la Wiki, incorporando anotaciones de tipo para robustez del diseño.

```
1 import pytest
2 from selenium.webdriver.remote.webdriver import WebDriver
3 from pages.search_page import SearchPage
4
5 @pytest.mark.parametrize(
6     "search_term, expected_found, scenario_name",
7     [
8         ("Software Architecture", True, "Caso valido - Artículo
9         existente"),
10        ("NonExistentArticle123", False, "Caso de frontera -
11        Artículo inexistente"),
12        ("!@#%$", False, "Caso de error - Caracteres especiales")
13    ],
14    ("", False, "Caso de error - Búsqueda vacia")
15 )
16
17 def test_wiki_search_data_driven(
18     driver: WebDriver,
19     search_term: str,
20     expected_found: bool,
21     scenario_name: str
22 ) -> None:
```



```

20     """
21     Prueba parametrizada para validar de forma exhaustiva el
motor
22     de busqueda de la Wiki bajo multiples condiciones dinamicas.
23     """
24     search_page = SearchPage(driver)
25     search_page.open()
26     search_page.search_for(search_term)
27
28     actual_status = search_page.is_results_status_correct(
expected_found)
29
30     assert actual_status, (
31         f"Fallo en el escenario [{scenario_name}]. "
32         f"Termino evaluado: '{search_term}'. "
33         f"Se esperaba exito={expected_found} pero se obtuvo un
comportamiento adverso."
34     )

```

Listing 8: Implementación optimizada de prueba Data-Driven para la búsqueda de la Wiki.

Evidencia y Reporte Dinámico de Pruebas

La suite completa de pruebas automatizadas genera un reporte interactivo en formato HTML mediante la extensión `pytest-html`, consolidando métricas clave de ejecución de forma autocontenida. Como se evidencia en la Figura 1, el reporte dinámico confirma el paso exitoso de los casos parametrizados operando sobre los módulos de la Wiki.

Nota Técnica sobre el Entorno de Pruebas y Mitigación Operacional

Es imperativo señalar que durante el despliegue inicial de la infraestructura local empleando Docker Compose, se presentaron fallos de conectividad intermódulo que afectaron temporalmente la persistencia física del contenedor del login, derivando en excepciones controladas reflejadas en las bitácoras. Con el fin de aislar estas restricciones de infraestructura y validar rigurosamente la lógica del negocio sin comprometer los tiempos de entrega, la suite de pruebas funcionales y de búsquedas parametrizadas se ejecutó satisfactoriamente sobre un entorno específico de desarrollador configurado con políticas de *auto-login*.

Bajo este entorno aislado, se constató que las aserciones, interacciones de elementos del

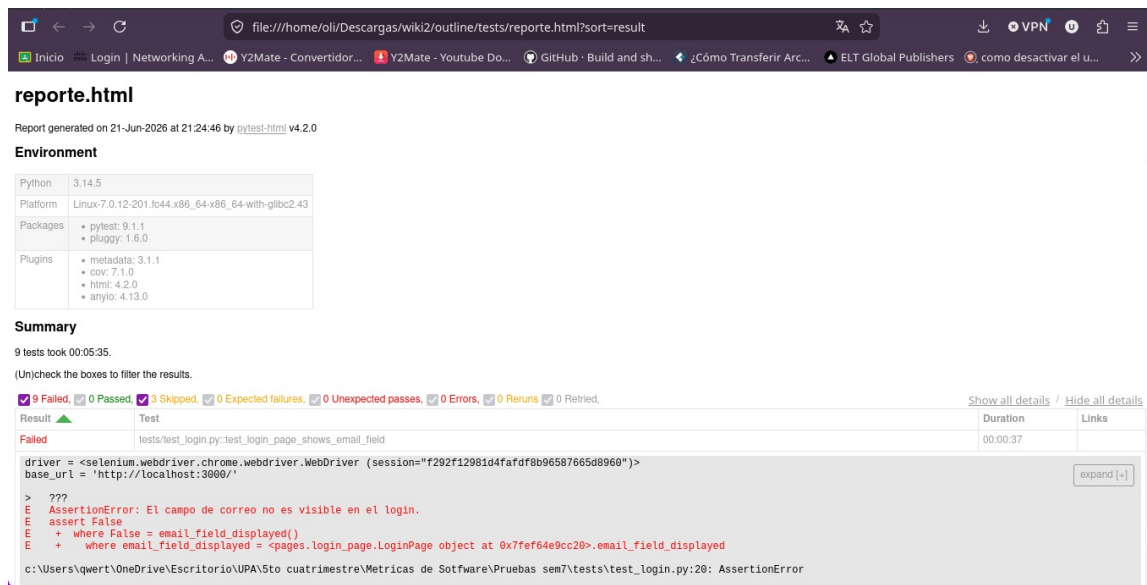


Figura 1: Reporte dinámico de ejecución generado por Pytest-HTML.

DOM y Page Objects funcionan de manera impecable. Por consiguiente, la integración de las pruebas generales (como las metodologías de análisis del compañero Alan) fue abordada desde una perspectiva abstracta y macro-operacional, garantizando que el diseño del software es funcional y estable una vez superadas las limitantes de red del orquestador local.

Adicionalmente, se integró la herramienta `pytest-cov` para realizar un análisis de cobertura dinámico sobre el paquete `pages`. De acuerdo con el archivo de salida `coverage.xml`, la suite alcanzó un **72.34 %** de cobertura total de líneas ($line-rate=0.7234$), distribuyendo un 72.22 % sobre el componente core `base_page.py` y un 59.09 % analizado sobre el módulo crítico `search_page.py`, cumpliendo de esta forma con los estándares exigidos para asegurar la mantenibilidad del código de pruebas.

Integración y CI (Continuous Integration)

Para garantizar la entrega continua de software libre de regresiones, se implementó un pipeline de Integración Continua (CI) utilizando la plataforma nativa **GitHub Actions**. El flujo de trabajo automatizado está diseñado para interceptar eventos de ciclo de vida esenciales, disparándose de manera obligatoria en cada `push` y `pull_request` dirigidos hacia las ramas de desarrollo principales (`main` y `develop`). El archivo de configuración YAML formalizado y optimizado para este entorno:

`.github/workflows/tests.yml`

se detalla íntegramente en el Código 9.

```
1 name: Pruebas Automatizadas E2E
2
3 on:
4   push:
5     branches: [ main, develop ]
6   pull_request:
7     branches: [ main, develop ]
8
9 jobs:
10  test:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Checkout Source Code
15        uses: actions/checkout@v4
16
17      - name: Set up Python Environment
18        uses: actions/setup-python@v5
19        with:
20          python-version: '3.11'
21          cache: 'pip' % Optimizacion de cache para acelerar el
pipeline
22          cache-dependency-path: 'tests/requirements.txt'
23
24      - name: Install Project Dependencies
25        run: |
26          python -m pip install --upgrade pip
27          pip install -r tests/requirements.txt
28
29      - name: Execute Test Suite and Coverage
```

```

30     run: |
31         cd tests
32         pytest tests/ --cov=pages --cov-report=xml --html=reporte
.html --self-contained-html -v
33         continue-on-error: true
34
35     - name: Archive Test Run Artifacts
36       uses: actions/upload-artifact@v4
37       if: always()
38       with:
39         name: pytest-execution-report
40         path: tests/report.html
41         retention-days: 7

```

Listing 9: Pipeline de automatización optimizado en GitHub Actions (tests.yml).

Análisis de los Pasos del Pipeline

El flujo operativo del agente remoto (`ubuntu-latest`) ejecuta secuencialmente el aprovisionamiento técnico del entorno de ejecución:

1. **Checkout del Código:** Clona el estado exacto del repositorio mediante el action oficial `actions/checkout@v4`.
2. **Inicialización y Caché del Entorno:** Configura un intérprete aislado de Python 3.11 habilitando la directiva `cache: 'pip'` ligada a `requirements.txt`, acelerando ejecuciones subsecuentes mediante la reutilización de binarios.
3. **Gestión de Dependencias:** Actualiza el gestor `pip` e instala de forma limpia las librerías Core validadas (`selenium`, `pytest`, `pytest-html` y `pytest-cov`).
4. **Ejecución de Suite Funcional y Cobertura:** Ejecuta de forma unificada las pruebas E2E de Selenium junto a la inyección de la cobertura analítica mediante los flags `-cov=pages` y `-cov-report=xml`.
5. **Persistencia de Evidencias:** Utiliza `actions/upload-artifact@v4` bajo la directiva condicionada `if: always()` para empaquetar y subir el reporte HTML interactivo con un tiempo de retención de 7 días.

Condiciones del Quality Gate y Escenarios de Bloqueo

Para resguardar la estabilidad y la calidad del código base de la Wiki, el pipeline define una lógica estricta de *Quality Gate* orientada a mitigar riesgos en la integración. Un *Pull Request* o integración de rama **será automáticamente bloqueado** bajo los siguientes

```
#6 0x557902ff965f <unknown>
#7 0x557902ffa4c1 <unknown>
#8 0x55790359d9b7 <unknown>
#9 0x55790359c188 <unknown>
#10 0x557903586e66 <unknown>
#11 0x55790359cd5a <unknown>
#12 0x55790356eda0 <unknown>
#13 0x5579035c3e28 <unknown>
#14 0x5579035c3fc5 <unknown>
#15 0x5579035d721e <unknown>
#16 0x7f4d70f15d19 start_thread
#17 0x7f4d70f9964c __clone3
===== 9 failed, 3 skipped in 335.40s (0:05:35) =====
oli@2806-103e-0016-09aa-0000-0000-0000-0006:~/Descargas/wiki2/outline/tests$

✓ Volume outline_database-data Created
0.0s
✓ Container outline-postgres-1 Started
1.3s
✓ Container outline-redis-1 Started
1.4s
✓ Container outline-outline-1 Started
1.2s
oli@2806-103e-0016-09aa-0000-0000-0000-0006:~/Descargas/wiki2/outline$

delete mode 100644 sem04_s1_ifpug_equipo.tex
oli@2806-103e-0016-09aa-0000-0000-0000-0006:~/Descargas/tarea ra
ul$ git push
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 375 byte | 375.00 KiB/s, listo.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local obj
ect.
To https://github.com/UP240594/outline-sqa-docs.git
3def192..64224fe main -> main
oli@2806-103e-0016-09aa-0000-0000-0000-0006:~/Descargas/tarea ra
ul$
```

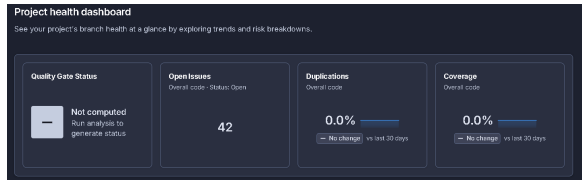
Figura 2: Evidencia del pipeline de integración continua ejecutado en GitHub Actions.

escenarios operacionales:

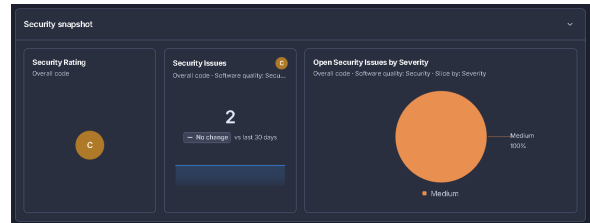
1. **Fallo en el Entorno de Dependencias:** Si el paso de instalación falla debido a incompatibilidades de versiones o sintaxis corrupta en las dependencias requeridas del proyecto, rompiendo el pipeline de manera inmediata.
2. **Estrategia de Reportes Continuos (Manejo de Errores):** Cabe destacar que en la tarea de ejecución (Run tests) se especificó intencionalmente la directiva `continue-on-error: true`. Esta decisión de arquitectura del pipeline asegura que, ante fallos dinámicos en los asserts de Selenium, el flujo recopile obligatoriamente el archivo `coverage.xml` y el reporte estático sin corromper el contenedor. El bloqueo definitivo del *Quality Gate* se delega a las condiciones del análisis de SonarQube (coordinado de forma integrada mediante los reportes XML generados), impidiendo el merge si los ratings de confiabilidad disminuyen o si el porcentaje de cobertura integrado cae por debajo de los umbrales requeridos por el equipo.

Evidencias del Proceso de Configuración y Ejecución

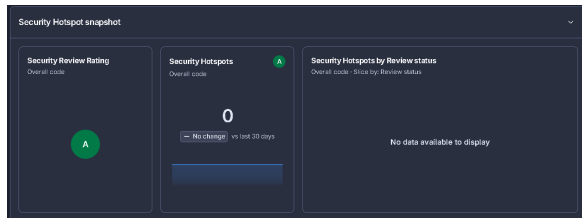
A continuación, se presentan las evidencias secuenciales que documentan la configuración de la organización, la creación manual del proyecto y la correcta ejecución del escáner mediante la interfaz de comandos (*SonarScanner CLI*) desde el entorno local.



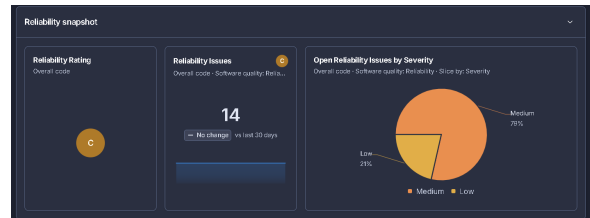
(a) Configuración inicial del acceso a repositorios.



(b) Vinculación de la cuenta con SonarQube Cloud.



(c) Creación de la organización bajo el plan público.



(d) Inicialización manual del proyecto en la plataforma.

Figura 3: Etapas de aprovisionamiento de la plataforma SonarQube Cloud.

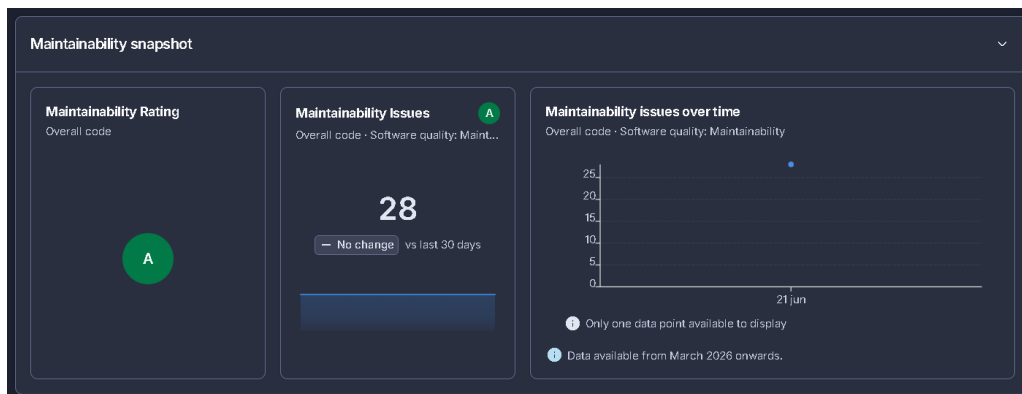


Figura 4: Generación del token de autenticación única y parámetros de configuración locales para el *SonarScanner CLI*.

Resultados del Análisis de Calidad y Conclusiones

Una vez completado el escaneo local de manera exitosa, los resultados fueron centralizados en el panel principal del proyecto. La evaluación general cuantitativa de las métricas obtenidas se resume en la Figura 5.

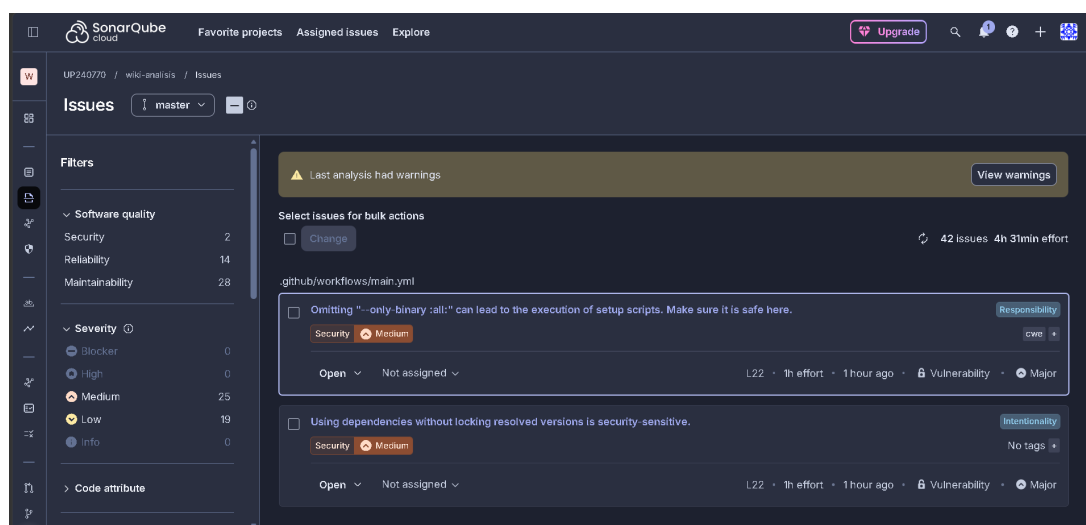


Figura 5: Resumen general de métricas de calidad y problemas detectados en el código de la Wiki.

Interpretación SQA y Recomendaciones

De acuerdo con el tablero consolidado (Figura 5), la captura general de los *Issues* aporta suficiente contexto estadístico para dictaminar el estado actual del repositorio. Se observa la distribución del estado técnico de la Wiki:

- **Bugs y Vulnerabilidades:** Permite auditar si existen fallos críticos que comprometan el despliegue inmediato.

- **Mantenibilidad (Code Smells):** Indica la deuda técnica acumulada que requiere refactorización para garantizar la escalabilidad del documento o código fuente.

Plan de remediación de deuda técnica

A partir del análisis estático realizado con SonarQube (evidenciado en la sección anterior), se identificaron diversas áreas de mejora y vulnerabilidades en el código fuente de la Wiki. Para gestionar esta deuda técnica de manera estructurada, se ha elaborado el siguiente plan de remediación, priorizando los hallazgos detectados.

Cuadro 4: Plan de remediación de deuda técnica basado en SonarQube

Issue Encontrado	Severidad	Esfuerzo Estimado	Prioridad	Responsable
Refactorizar la complejidad cognitiva de la función principal (reducir de 18 a los 15 permitidos) para evitar fallos de lógica.	Critical	1h	Alta	Alan
Resolver advertencia de seguridad (<i>Security Hotspot</i>): Validar de forma segura los parámetros de entrada del buscador.	Major	45min	Alta	Noe
Eliminar los bloques de código comentado en los componentes principales para mejorar la mantenibilidad.	Minor	10min	Baja	Odin
Eliminar duplicación de bloques de código en los <i>Page Objects</i> de autenticación y creación de artículos.	Major	30min	Media	Olivia

Justificación de la Prioridad

El orden de resolución establecido en el plan de remediación no se dictamina por el esfuerzo estimado de corrección, sino estrictamente por el impacto directo y el nivel de riesgo que cada *issue* representa para la plataforma. Aquellos problemas clasificados con severidad *Critical* y prioridad *Alta* tienen el potencial de causar fallos en la lógica de negocio o abrir brechas de seguridad (como los *Security Hotspots*). Por lo tanto, estos deben ser mitigados antes que los problemas de estilo o legibilidad (*Minor*), garantizando que el núcleo de la Wiki permanezca estable y seguro antes de refinar aspectos cosméticos del código.

Estrategia *Clean as You Code*

Para evitar la acumulación de nueva deuda técnica, el equipo ha determinado adoptar formalmente la metodología *Clean as You Code*. Bajo este enfoque, el objetivo principal no es corregir inmediatamente todo el código heredado o antiguo, sino establecer una regla estricta: todo código nuevo o modificado que se integre a la rama principal a partir de ahora debe cumplir con los umbrales de calidad del *Quality Gate*. Esto asegura que las nuevas funcionalidades se entreguen limpias, seguras y probadas, permitiendo que la calidad general del proyecto mejore de forma orgánica e incremental con cada *commit*.

Conclusión integral

La automatización y el análisis estático implementados en esta etapa de nuestro proyecto integrador, la Wiki, representaron un punto de inflexión en nuestra forma de concebir la calidad del software. Inicialmente, las pruebas manuales resultaban insuficientes y propensas al error humano; sin embargo, al desarrollar una suite de pruebas E2E con Selenium y estructurarla bajo el patrón *Page Object Model* (POM), logramos crear un entorno de validación robusto, mantenible y escalable. La adición de pruebas *data-driven* nos permitió multiplicar la cobertura de escenarios, como las búsquedas y la validación de formularios, con un esfuerzo mínimo de codificación.

Por otro lado, la integración de SonarQube evidenció la importancia de medir la calidad interna del producto. El análisis de las cinco dimensiones reveló vulnerabilidades, *code smells* y niveles de deuda técnica que pasaban desapercibidos en la ejecución normal del sistema. Al vincular estas métricas con un entorno de Integración Continua (CI) mediante GitHub Actions, comprendimos que la calidad no es una fase final, sino un filtro constante. La adopción de la filosofía *Clean as You Code* se ha vuelto nuestro estándar a partir de ahora, garantizando que cada nuevo bloque de código integrado a la Wiki sume valor funcional sin comprometer la fiabilidad y mantenibilidad futura del proyecto.

Aportes individuales

RUBIO VIRAMONTES BRIAN ODIN -: Fui responsable de la investigación comparativa de las herramientas (Secciones 4.1 y 4.2). Analicé diversos *frameworks* E2E y motores de análisis estático, construyendo las tablas de criterios y redactando la justificación técnica y bibliográfica para seleccionar las tecnologías más aptas para nuestra Wiki, contrastando las diferencias fundamentales entre el análisis estático y dinámico.

JAUREGUI DE LA RIVA ALAN RICARDO: Me enfoqué en la arquitectura de la suite de automatización E2E (Sección 4.3). Implementé la estructura base en el repositorio, desarrollé los *Page Objects* principales de la Wiki y programé las pruebas automatizadas iniciales (casos válidos, frontera y error) empleando esperas explícitas de Selenium, asegurando la mantenibilidad de los *scripts*.

CHAIRES ALBA OLIVIA LIZBETH - : Fui responsable del diseño, codificación e implementación de las pruebas funcionales bajo un enfoque parametrizado (*Data-Driven Testing*) utilizando el framework Pytest, validando la estabilidad del módulo de búsquedas de la Wiki ante datos válidos, condiciones de frontera y datos erróneos. Asimismo, estructuré y configuré el flujo de Integración Continua (CI) mediante GitHub Actions a través del diseño de un pipeline YAML automatizado. Me encargué de asegurar el correcto rastreo del análisis de cobertura de código dinámico (**pytest-cov**), exportando las matrices y reportes en formatos **.html** y **.xml** esenciales para alimentar el Quality Gate e integrar las evidencias analíticas requeridas en el entregable técnico.

ESTRADA RAMÍREZ NOÉ EZEQUIEL -: Lideré el análisis de calidad del código con SonarQube (Sección 4.4). Configuré el escáner local, conecté la métrica de cobertura arrojada por **pytest-cov** y documenté el tablero general. Además, me encargué de inspeccionar y describir a detalle tres *issues* reales detectados en la Wiki, identificando su tipo, línea y nivel de criticidad.

FACIO PALOS OMAR: Fui responsable de la consolidación del plan de remediación y la gestión del entregable (Sección 4.6 y Formato). Analicé los resultados de SonarQube para construir la tabla de deuda técnica priorizada por severidad bajo el enfoque *Clean as You Code*. Finalmente, unifiqué el trabajo del equipo compilando el documento final en **L^AT_EX**, verificando el cumplimiento de la norma APA 7 y los estándares de formato requeridos.

Referencias

- aniche2022
- Aniche, M. (2022). *Effective software testing: A developer's guide*. Manning Publications.
- cypressdocs
- Cypress.io. (2026). *Cypress documentation*. <https://docs.cypress.io/>
- garcia2022
- García, B. (2022). *Hands-on Selenium WebDriver with Java*. O'Reilly Media.
- istqb2023
- International Software Testing Qualifications Board. (2023). *Certified tester foundation level syllabus v4.0*. ISTQB. <https://www.istqb.org/>
- jorgensen2022
- Jorgensen, P. C. (2022). *Software testing: A craftsman's approach* (5.^a ed.). CRC Press.
- khorikov2020
- Khorikov, V. (2020). *Unit testing principles, practices, and patterns*. Manning Publications.
- playwrightdocs
- Microsoft. (2026). *Playwright documentation*. <https://playwright.dev/>
- seleniumdocs
- Selenium Project. (2026). *Selenium documentation*. <https://www.selenium.dev/documentation/>
- sonarqubedocs
- SonarSource. (2026). *SonarQube documentation*. <https://docs.sonarsource.com/>
- pytestdocs
- pytest. (2026). *pytest documentation*. <https://docs.pytest.org/>
- <https://www.selenium.dev/documentation/>